

computer graphics

TABLE OF CONTENTS

1. Introduction
2. Proposal
3. Schematic Diagram
4. List of Objects
5. Functions to Represent the Objects
6. Interactive Functions
7. Task Assignment and Codes of Functions
8. Conclusion

1. INTRODUCTION

This is our Computer Graphics lab project. We made it using OpenGL and C++ with the GLUT library. The project is called "**Maritime World Journey**" and it shows a ship travelling through four different scenes.

Each scene has its own environment with different objects, colors, and animations. The user can interact with the program using the keyboard and mouse — things like switching between day and night, turning on rain, starting a train, or launching fighter jets.

We used basic 2D graphics concepts like drawing shapes with `GL_POLYGON` and `GL_QUADS`, moving objects using timer functions, and handling keyboard and mouse input. This project helped us understand how a real graphics program is built from scratch.

2. PROPOSAL

Our idea was to simulate a ship's journey through four different places. The ship moves from one scene to the next on its own, and the user can also interact with each scene separately.

Scene 1 – Harbor Town: A city near the sea with mountains, a lighthouse, a big building, a bridge, lamp posts, and two moving cars. The ship starts here.

Scene 2 – Countryside: Green hills, a road with cars, a train on a railway, trees, a beach, and water. Rain can be turned on here.

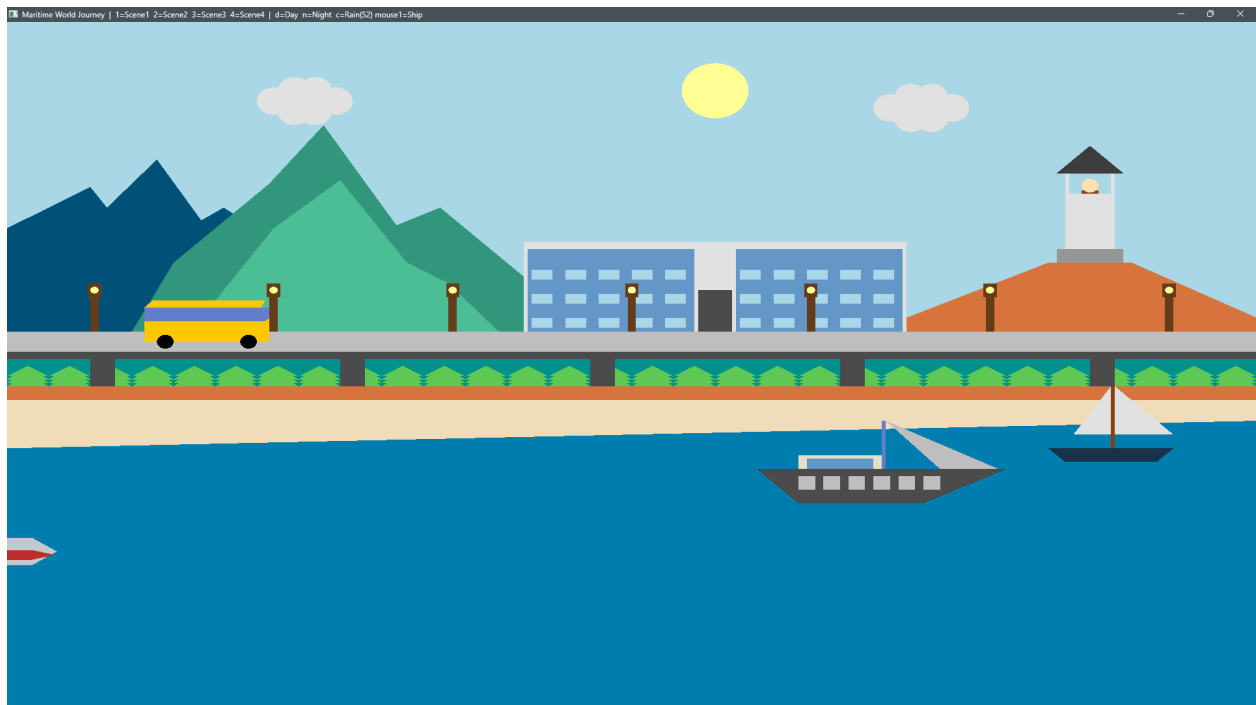
Scene 3 – Wind Farm: Two spinning wind turbines, two houses, a road with a bus and a car, and fighter jets that fly when a key is pressed.

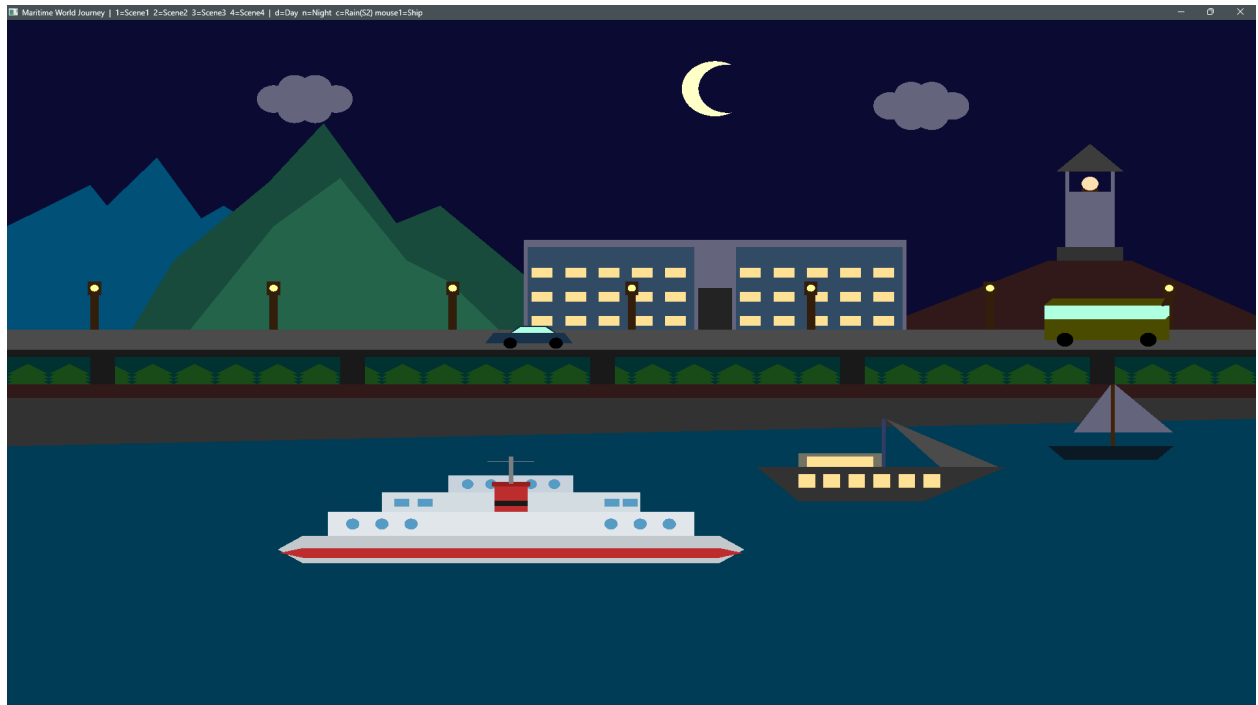
Scene 4 – Beach: A calm beach with clouds, a flying bird, two boats on the water, and a cliff. The ship parks here at the end of the journey.

The user can switch scenes with keys 1–4, change day/night with 'd' and 'n', and use mouse clicks to control the ship, train, and cars.

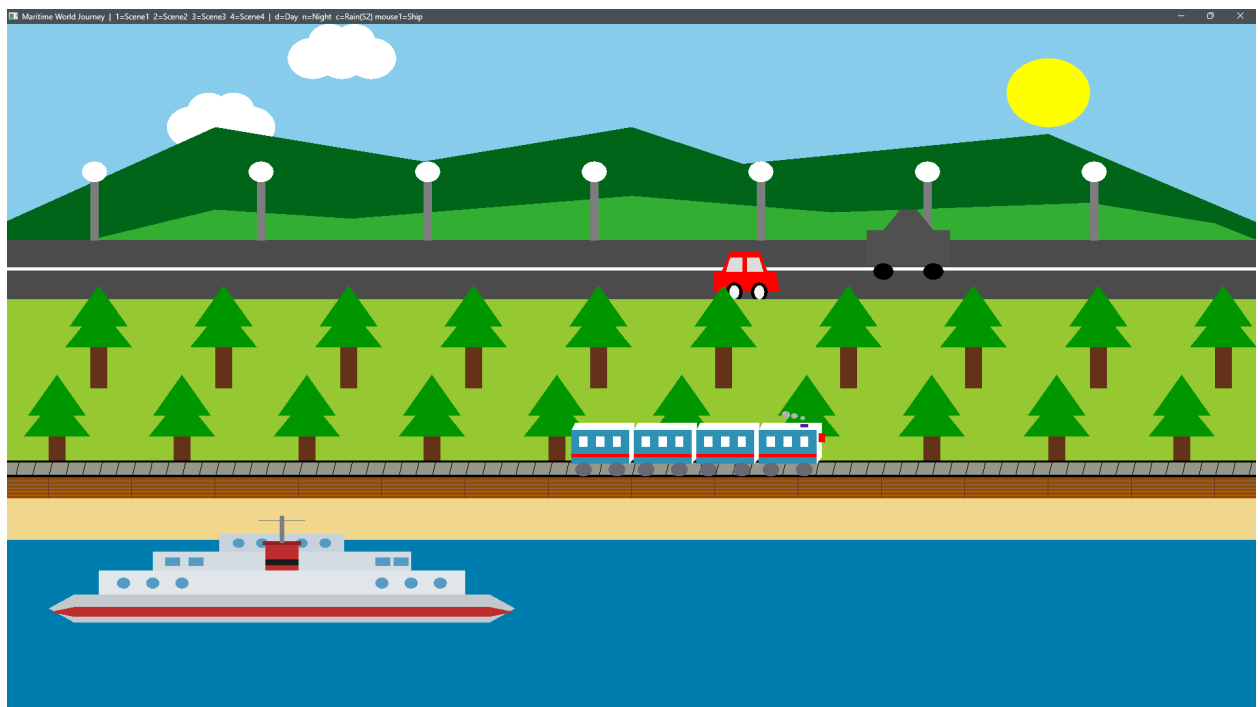
3. SCHEMATIC DIAGRAM

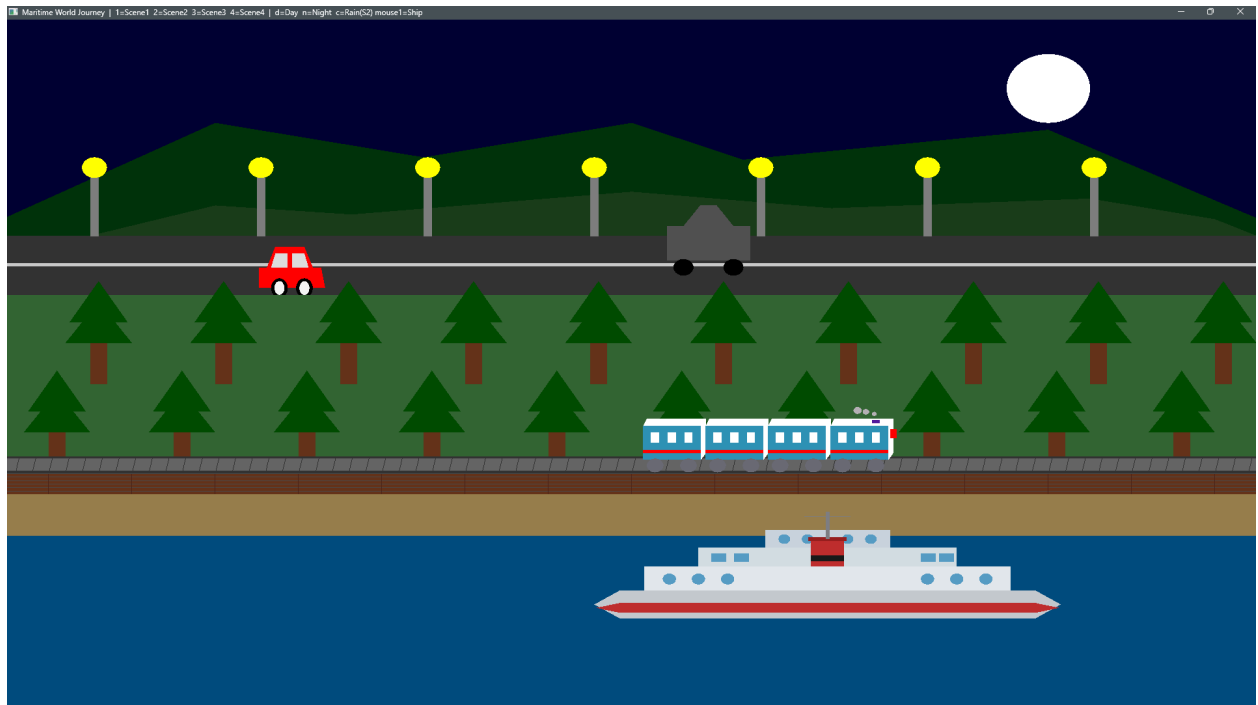
Scenario -1



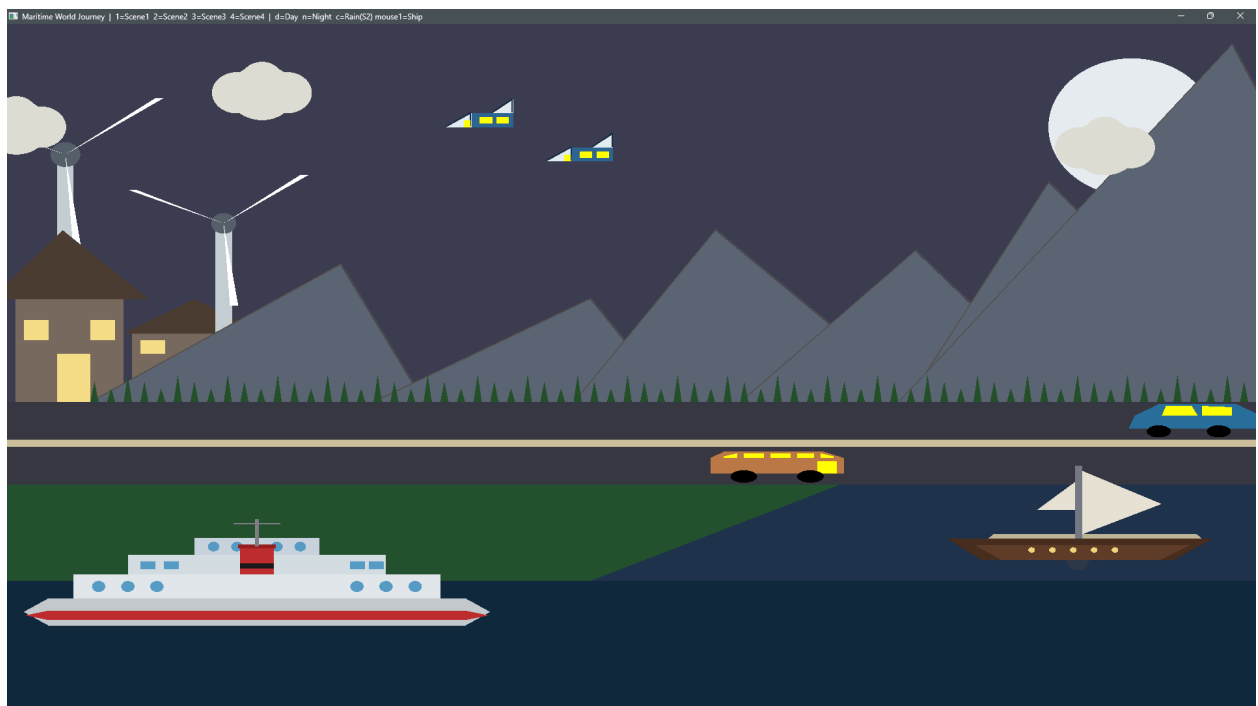
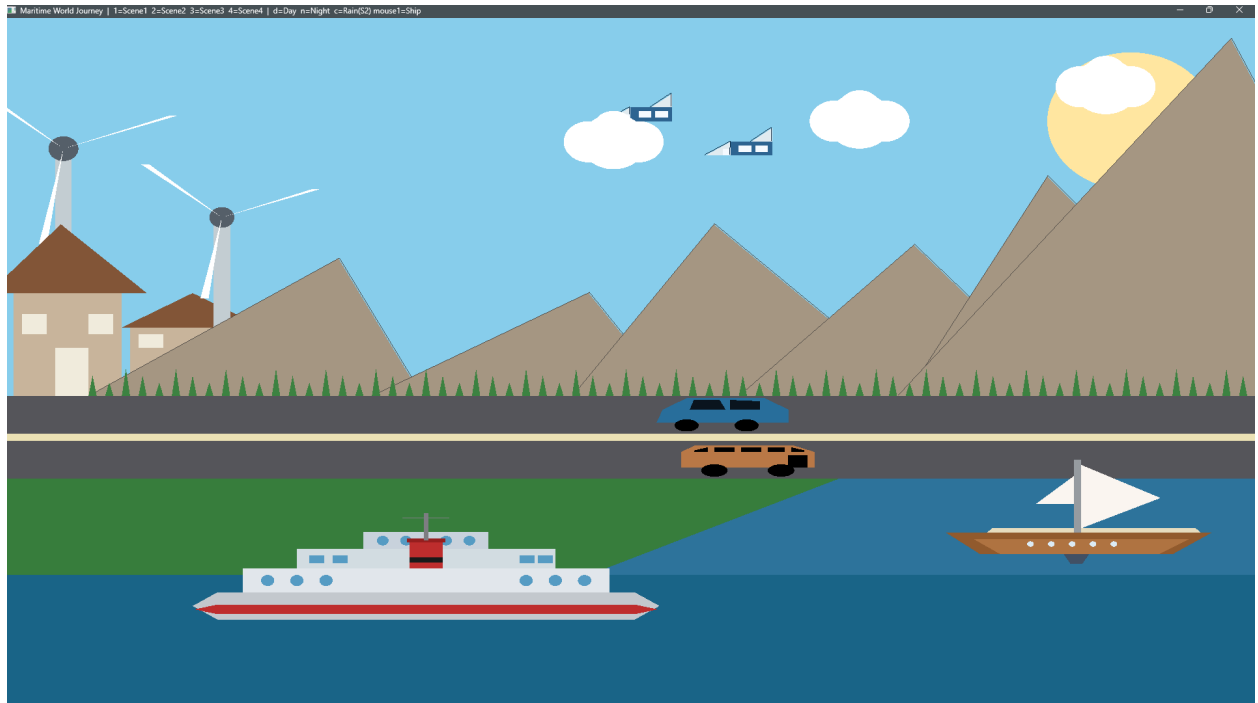


Scenario -2

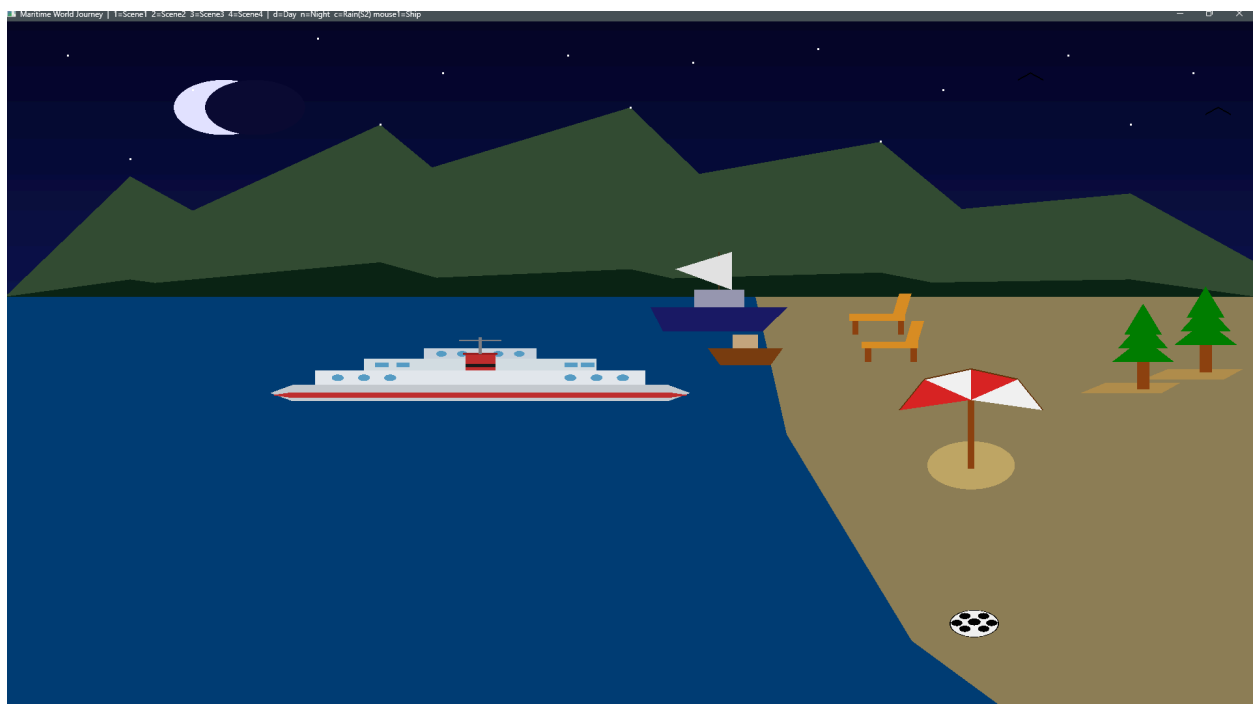
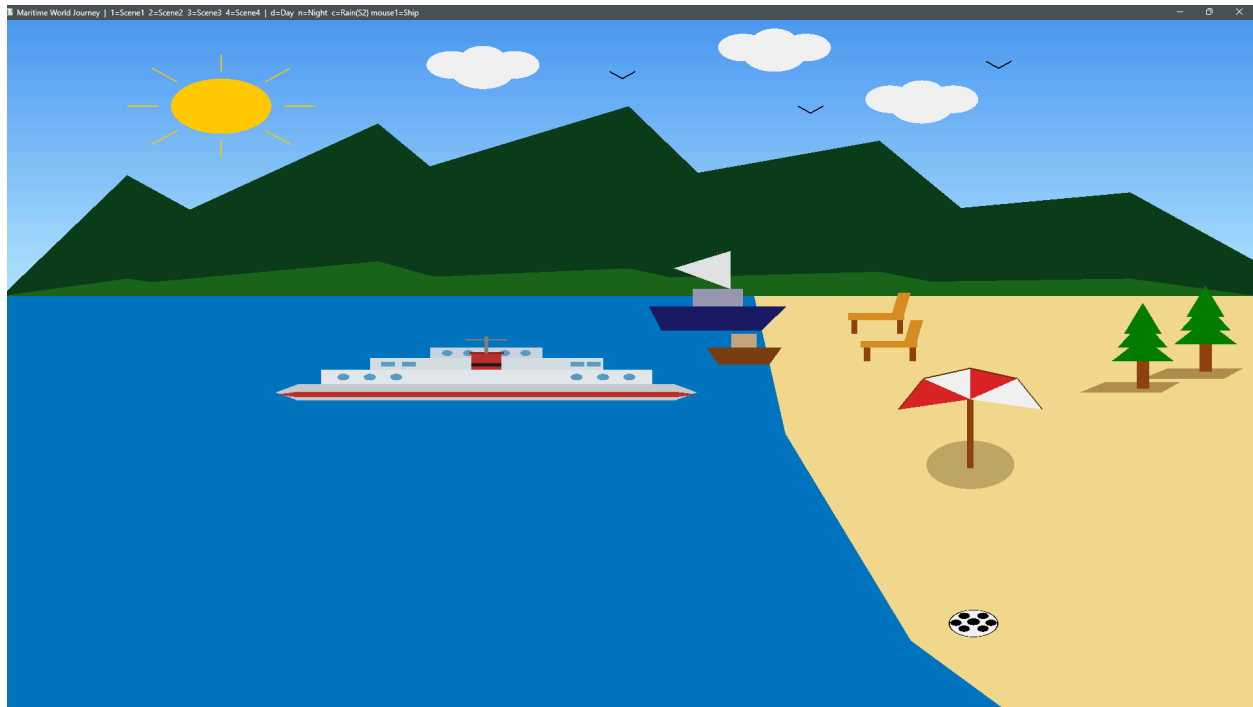




Scenario -3



Scenario -4



4. LIST OF OBJECTS

#	Object	Scene	Animated?
1	Sky	All	Color changes (day/night/rain)
2	Sun	1, 2, 3	No
3	Moon (crescent)	1, 2, 3	No
4	Stars	4 (night)	No
5	Clouds	All	Yes – drift sideways
6	Mountains	1	No
7	Rolling Hills	2	No
8	Sandy Hills	3	No
9	Large Building (with windows)	1	No
10	Lighthouse	1	No
11	Bridge Road	1	No
12	Lamp Posts	1, 2	Glow at night
13	Wind Turbine (tall)	3	Yes – blades spin
14	Wind Turbine (short)	3	Yes – blades spin
15	House 1 (small)	3	No
16	House 2 (large)	3	No
17	Ship (large)	1, 2, 3	Yes – moves across screen
18	Ship (small, scaled)	4	Yes – parks at shore
19	Yellow Car	1	Yes – moves left
20	Blue Car	1, 3	Yes – moves left
21	Grey Car	2	Yes – moves left
22	Red Car	2	Yes – moves right
23	Orange Bus	3	Yes – moves right
24	Train (engine + 3 cars)	2	Yes – mouse toggle
25	Fighter Jet 1	3	Yes – key activated
26	Fighter Jet 2	3	Yes – key activated

#	Object	Scene	Animated?
27	Sailboat (static)	1	No
28	Speedboat (static)	1	No
29	Animated Boat 1	4	Yes – moves and reverses
30	Animated Boat 2	4	Yes – moves and stops
31	Bird	4	Yes – flies + wing flap
32	Beach / Sand	2, 4	No
33	Water / Sea	1, 2, 3, 4	No (color changes)
34	Grass	1, 2, 3	No
35	Trees	2	No
36	Railway Track	2	No
37	Dam / Wall	2	No
38	Cliff	4	No
39	Rain Effect	2	Yes – random streaks

5. FUNCTIONS TO REPRESENT THE OBJECTS

Shared Functions (Used in Multiple Scenes)

`circle()` — Draws a filled circle. Takes the radius, center position, and color as input. Used for the sun, moon, clouds, portholes, lamp bulbs, and wheels all across the project.

`circle3()` — Another circle function but uses `GL_TRIANGLE_FAN`. Used in Scene 3 for clouds and wheels.

`oval2()` — Draws an ellipse. Used for train wheels and car wheels in Scene 2.

Ship Functions

`drawShipShape()` — Draws the full detailed ship with hull, red stripe, two decks, a bridge, portholes, funnel, mast, and radar bar. All coordinates are centered at the origin so the ship can be placed anywhere.

`drawShip(x, y)` — Places the ship at position (x, y) using `glTranslatef`. Used in Scenes 1, 2, and 3.

`drawShip4(x, y)` — Same as `drawShip()` but also scales the ship down using `glScalef(0.0012f)` so it looks small and fits Scene 4's beach environment.

Scene 1 Functions

`Scene1()` — Draws everything in Scene 1: sky, sun/moon, clouds, three mountain layers, a coastal city building with grid windows, a lighthouse, a bridge with pillars, 10 lamp posts, two moving cars, water, a sailboat, a speedboat, and the ship.

Scene 2 Functions

`drawCloud2(x, y)` — Draws a cloud made of 5 circles. Turns dark when rain is on. Disappears at night.

`drawLamp2(x, y)` — Draws a street lamp. The bulb turns yellow at night or during rain.

`drawTrain2()` — Draws a full train: one engine and three passenger cars, each with windows, a red stripe, wheels, and couplers. The engine also has a chimney with smoke puffs.

`Scene2()` — Draws everything in Scene 2: sky, sun/moon, two hill layers, road, trees in two rows, railway, train, lamp posts, beach, water, two moving cars, a dam wall, clouds, and rain streaks if rain is active.

Scene 3 Functions

`drawBlade3(cx, cy, length, angle)` — Draws one turbine blade as a triangle pointing outward at the given angle. Called three times per turbine, 120° apart, to make the full spinning rotor.

`drawCloud3(x, y)` — Draws a cloud for Scene 3. Color turns grey at night.

`drawCar3()` — Draws the blue car in Scene 3 with body, windows (yellow at night), and wheels.

`drawBus3()` — Draws the orange bus with body, five windows (yellow at night), and wheels.

`reset3()` — Resets everything in Scene 3 back to starting positions: car, bus, turbine angle, clouds, and jets.

Scene3() — Draws everything in Scene 3: sky, two jets, two houses, two wind turbines, sun/moon, hills, clouds, grass tufts, road, bus, car, water, field, a small boat, and the ship.

Scene 4 Functions

drawCloud4(x, y) — Draws a Scene 4 cloud using 4 overlapping circles from the shared **circle()** function.

Scene4() — Draws everything in Scene 4: sky, sun or stars (night), three drifting clouds, sea, beach, cliff, two animated boats, a flapping bird made with **GL_LINES**, and the small scaled-down ship.

6. INTERACTIVE FUNCTIONS

Keyboard Controls

Key	Action
1	Go to Scene 1
2	Go to Scene 2
3	Go to Scene 3
4	Go to Scene 4
d	Switch all scenes to Day mode
n	Switch all scenes to Night mode
c (Scene 2 only)	Toggle Rain on/off
r (Scene 3 only)	Reset all Scene 3 objects
Any key (Scene 3)	Activate fighter jets
ESC	Exit the program

Mouse Controls

Scene	What Clicking Does
Scene 1	Starts the ship moving right
Scene 2	Starts the ship + toggles the train on/off
Scene 3	Starts the ship + starts the blue car

Scene	What Clicking Does
Scene 4	1st click: ship moves to shore and parks. 2nd click: ship returns back

7. TASK ASSIGNMENT AND CODES OF FUNCTIONS

7.1 Task Assignment

Member	What They Did
Member 1	Scene 1, ship drawing functions, shared <code>circle()</code> , <code>shipTimer()</code> , report
Member 2	Scene 2, train, lamp posts, clouds, rain, <code>timer2()</code>
Member 3	Scene 3, turbines, car, bus, jets, <code>reset3()</code> , <code>timer3()</code> , keyboard handler
Member 4	Scene 4, bird, boats, clouds, <code>mouse()</code> handler, <code>main()</code> setup, all timer registration

7.2 Important Code Snippets

`circle()` — Used everywhere to draw circular shapes

```
void circle(float radius, float xc, float yc, float r, float
g, float b)
{
    glBegin(GL_POLYGON);
    glColor3f(r, g, b);
    for (int i = 0; i < 200; i++) {
        float angle = (i * 2 * PI) / 200;
        glVertex2f(radius * cos(angle) + xc, radius * sin(ang
le) + yc);
    }
    glEnd();
}
```

We pass the color as parameters so we can reuse this one function for many different objects — sun, moon, portholes, clouds, wheels, and lamp bulbs.

`drawBlade3()` — Draws one wind turbine blade

```
void drawBlade3(float cx, float cy, float length, float angle)
{
    float rad = angle * PI / 180.0f;
    float x1 = cx + length * cos(rad);
    float y1 = cy + length * sin(rad);
    glBegin(GL_TRIANGLES);
    glVertex2f(cx, cy);
    glVertex2f(x1 - 5, y1);
    glVertex2f(x1 + 5, y1);
    glEnd();
}
```

This is called 3 times per turbine with angles 0°, 120°, and 240°. Since `blade3Angle` increases by 4° every timer tick, all three blades rotate together.

`shipTimer()` — Moves the ship and switches scenes automatically

```
void shipTimer(int value)
{
    if (shipMoving) {
        if (currentScene == 4)
            shipX4 += (float)shipDir * shipSpeed4;
        else
            shipX += (float)shipDir * shipSpeed;

        if (currentScene == 1 && shipX > 1600.0f) {
            currentScene = 2;
            resetShipForScene(2);
        }
        else if (currentScene == 2 && shipX < -220.0f) {
            currentScene = 3;
        }
    }
}
```

```

        resetShipForScene(3);
    }
    else if (currentScene == 3 && shipX > 1600.0f) {
        currentScene = 4;
        resetShipForScene(4);
    }
    glutPostRedisplay();
}
glutTimerFunc(16, shipTimer, 0);
}

```

When the ship goes past the screen edge, `currentScene` updates and the ship resets to the correct starting position for the next scene. This is what makes the whole journey feel connected.

`keyboard()` — Handles all key presses

```

void keyboard(unsigned char key, int, int)
{
    if (key == '1') { currentScene=1; resetShipForScene(1); g
lutPostRedisplay(); return; }
    if (key == '2') { currentScene=2; resetShipForScene(2); g
lutPostRedisplay(); return; }
    if (key == '3') { currentScene=3; resetShipForScene(3); g
lutPostRedisplay(); return; }
    if (key == '4') { currentScene=4; resetShipForScene(4); g
lutPostRedisplay(); return; }

    if (key == 'n') { night1=true; night2=true; night3=1; d
ay4=0; glutPostRedisplay(); }
    if (key == 'd') { night1=false; night2=false; night3=0; d
ay4=1; glutPostRedisplay(); }

    if (key == 'c' && currentScene == 2) { rain2 = !rain2; gl
utPostRedisplay(); }
    if (key == 'r' && currentScene == 3)    reset3();
}

```

```

    if (currentScene == 3 && key != 27)    jets30n = 1;

    if (key == 27) exit(0);
}

```

`mouse()` — Handles mouse clicks

```

void mouse(int button, int state, int, int)
{
    if (button == GLUT_LEFT_BUTTON && state == GLUT_DOWN) {

        if (currentScene == 4) {
            if (!shipMoving) {
                ship4TripCount++;
                if (ship4TripCount % 2 == 1) {
                    shipDir = 1;
                    shipX4 = -1.2f;
                } else {
                    shipDir = -1;
                    shipX4 = 0.18f;
                }
                shipMoving = true;
            }
            return;
        }

        shipMoving = true;
        if (currentScene == 2)                train20n = !train
20n;
        if (currentScene == 3 && !car30n)    car30n = 1;
    }
}

```

8. CONCLUSION

We finished building the Maritime World Journey project and it works the way we planned. The ship travels through all four scenes, the day/night system works across all scenes at once, and each scene has its own unique objects and interactions.

The most difficult part was the `shipTimer()` function because we had to handle two different coordinate systems — the normal 0 to 1500 range used in Scenes 1, 2, and 3, and the much smaller -1.2 to 1.2 range in Scene 4. Getting the ship to look right in both systems took some effort.

We also learned that organizing code into separate functions for each object makes things a lot easier to manage. For example, the shared `circle()` function is used more than 30 times in the whole project, which saved us from writing the same math over and over.
